

Ứng dụng phần mềm giải bài toán Tối ưu hóa

Trường Công nghệ thông tin Phenikaa

Đại học Phenikaa

Ứng dụng phần mềm trong tối ưu hóa

- 1 Tối ưu hóa với Python
 - CVXPY
 - SciPy
 - PuLP
- 2 Tối ưu hóa với MATLAB
- 3 Một số phần mềm khác

1. Tối ưu hóa với Python

Xét bài toán:

$$\min_x f(x)$$

Với các ràng buộc:

$$h_i(x) = 0 \quad (\text{Ràng buộc đẳng thức})$$

$$g_j(x) \leq 0 \quad (\text{Ràng buộc bất đẳng thức})$$

$$x_k^L \leq x_k \leq x_k^U \quad (\text{Ràng buộc biên})$$

Các thư viện Python phổ biến

Thư viện cơ bản:

- SciPy (minimize, linprog)
- PuLP (Quy hoạch tuyến tính)

Thư viện nâng cao:

- CVXPY (Tối ưu lồi)
- Pyomo (Mô hình hóa phức tạp)
- GEKKO (Tối ưu động)

So sánh các thư viện

Thư viện	Phù hợp nhất cho
CVXPY	Bài toán lồi
Pyomo	Mô hình phức tạp
SciPy	Mục đích chung
PuLP	LP/MILP đơn giản

- Thư viện Python mã nguồn mở cho **tối ưu hóa lồi**
- Đặc điểm nổi bật:
 - Cú pháp trực quan, gần với toán học
 - Hỗ trợ nhiều loại bài toán (LP, QP, SDP,...)
 - Tự động kiểm tra tính lồi (DCP rules)

Giới thiệu CVXPY

- Thư viện Python mã nguồn mở cho **tối ưu hóa lồi**
- Đặc điểm nổi bật:
 - Cú pháp trực quan, gần với toán học
 - Hỗ trợ nhiều loại bài toán (LP, QP, SDP,...)
 - Tự động kiểm tra tính lồi (DCP rules)

Giới thiệu CVXPY

- Thư viện Python mã nguồn mở cho **tối ưu hóa lồi**
- Đặc điểm nổi bật:
 - Cú pháp trực quan, gần với toán học
 - Hỗ trợ nhiều loại bài toán (LP, QP, SDP,...)
 - Tự động kiểm tra tính lồi (DCP rules)
- Ứng dụng:
 - Machine Learning
 - Tài chính định lượng
 - Điều khiển tối ưu

Các loại bài toán hỗ trợ

Loại bài toán	Cú pháp CVXPY
Linear Program (LP)	<code>cp.Minimize(c.T@x)</code>
Quadratic Program (QP)	<code>cp.quad_form(x, P)</code>
Semidefinite Program (SDP)	<code>cp.variable((n,n), PSD=True)</code>
Sparse Optimization	<code>cp.sum_entries(cp.abs(x))</code>

Lưu ý

CVXPY tự động kiểm tra tính lồi thông qua **DCP (Disciplined Convex Programming)**

Cấu trúc một bài toán CVXPY

```
1      import cvxpy as cp
2
3      # Variable declaration
4      x = cp.Variable(n)
5
6      # Constraints
7      constraints = [f(x) <= 0, ...]
8
9      # Objective
10     objective = cp.Minimize(cp.norm(A@x - b))
11
12     # Solve the problem
13     prob = cp.Problem(objective, constraints)
14     prob.solve()
```

Ví dụ: Bài toán bình phương tối thiểu

Phát biểu bài toán:

Tối thiểu hóa $\|Ax - b\|_2^2$ (tổng bình phương sai số)

Tìm nghiệm tối ưu tương ứng x^* , trong đó $r = Ax^* - b$ được gọi là phần dư (residual).

Ví dụ: Bài toán bình phương tối thiểu

```
1      import cvxpy as cp
2      import numpy as np
3
4      # Data
5      m, n = 30, 20
6      np.random.seed(1)
7      A = np.random.randn(m, n)
8      b = np.random.randn(m)
9      # Variable and Problem
10     x = cp.Variable(n)
11     objective = cp.Minimize(cp.sum_squares(A @ x
12                                     - b))
13     constraints = [0 <= x, x <= 1]
14     prob = cp.Problem(objective, constraints)
15     # Solve the problem
16     result = prob.solve()
17     print(result) # Output: 19.83126370644502
```

Ví dụ: Bài toán tối ưu hóa danh mục đầu tư

Bài toán

Tối đa hóa lợi nhuận điều chỉnh rủi ro:

$$\text{maximize } \mu^T w - \gamma w^T \Sigma w$$

với ràng buộc:

$$1^T w = 1, \quad w \geq 0$$

- μ : vector lợi nhuận kỳ vọng
- Σ : ma trận hiệp phương sai
- γ : tham số ưu tiên rủi ro

Ví dụ: Bài toán tối ưu hóa danh mục đầu tư

```
1      import numpy as np
2      import scipy.sparse as sp
3      import cvxpy as cp
4
5      # Generate random data
6      np.random.seed(1); n = 10
7      mu = np.abs(np.random.randn(n, 1))
8      Sigma = np.random.randn(n, n)
9      Sigma = Sigma.T.dot(Sigma)
```

Ví dụ: Bài toán tối ưu hóa danh mục đầu tư

```
1      # Solve the problem using CVXPY
2      w = cp.Variable(n)
3      gamma = cp.Parameter(nonneg=True)
4      ret = mu.T @ w
5      risk = cp.quad_form(w, Sigma)
6      objective = cp.Maximize(ret - gamma * risk)
7      constraints = [cp.sum(w) == 1, w >= 0]
8      prob = cp.Problem(objective, constraints)
9      gamma.value = 0.2
10     prob.solve()
11     # Output: 1.5000976202809573
```

Ví dụ: Bài toán tối ưu lồi

```
1      import cvxpy as cp
2
3      # Define variables
4      x = cp.Variable()
5      y = cp.Variable()
6
7      # Define constraints
8      constraints = [x + y == 1, x - y >= 2]
9
10     # Define the objective function
11     objective = cp.Minimize(x**2 + y**2)
12
13     # Formulate the problem
14     prob = cp.Problem(objective, constraints)
```

Ví dụ: Bài toán tối ưu lồi

```
1      # Solve the problem
2      result = prob.solve()
3      print(f"Optimal value: {result}")
4      print(f"x: {x.value}, y: {y.value}")
5
6      # >>> Optimal value: 2.5
7      # x: 1.5, y: -0.5000000000000001
```

Ưu điểm của CVXPY

- **Dễ sử dụng**

- Cú pháp gần với toán học
- Không cần chuyển đổi sang dạng chuẩn

- **Linh hoạt**

- Hỗ trợ nhiều loại bài toán
- Dễ dàng kết hợp với NumPy

- **Mạnh mẽ**

- Giải được bài toán quy mô lớn
- Nhiều solver backend (ECOS, SCS,...)

Các ứng dụng khác

- **Xử lý ảnh:**

- Khôi phục ảnh (inpainting)
- Giảm nhiễu

Original Image



Corrupted Image



In-Painted Image



Difference Image



- **Điều khiển tối ưu:**

- Lập quỹ đạo tên lửa
- Điều khiển robot



Giới thiệu SciPy Optimization

- Thư viện mạnh mẽ trong hệ sinh thái SciPy
- Hỗ trợ nhiều loại bài toán tối ưu:
 - Tối ưu không ràng buộc/ràng buộc
 - Tìm nghiệm (root-finding)
 - Curve fitting
- Ưu điểm:
 - Dễ sử dụng
 - Tích hợp nhiều thuật toán (BFGS, Nelder-Mead, SLSQP...)

Các hàm chính trong scipy.optimize

Các hàm tiêu biểu

- `minimize()`: Tối thiểu hóa hàm mục tiêu
- `least_squares()`: Bình phương tối thiểu
- `linprog()`: Tối ưu tuyến tính
- `minimize-scalar()`: Tối ưu hàm một biến

minimize: Tối ưu không ràng buộc

Ví dụ cú pháp minimize

```
from scipy.optimize import  
    minimize  
result = minimize(fun, x0, method  
                  = 'BFGS')
```

minimize: Tối ưu không ràng buộc

Bài toán

Tìm cực tiểu hàm $f(x) = x_0^2 + x_1^2$

```
def objective(x):  
    return x[0]**2 + x[1]**2  
  
result = minimize(objective, [1,  
    1], method='BFGS')  
print(result.x) # Output: [~0.0,  
    ~0.0]
```

- $x_0=[1, 1]$: Điểm khởi tạo
- `method='BFGS'`: Thuật toán gradient-based
- Kết quả gần đúng $[0.0, 0.0]$

minimize với ràng buộc

- Ràng buộc tuyến tính hoặc phi tuyến
- Phương pháp: SLSQP, trust-constr

Ràng buộc đoạn

Ví dụ với ràng buộc $0 \leq x_0 \leq 1; -0.5 \leq x_1 \leq 2.0$

```
from scipy.optimize import Bounds
bounds = Bounds([0, -0.5], [1.0,
                           2.0])
```

minimize với ràng buộc

Ràng buộc tuyến tính: Các ràng buộc $x_0 + 2x_1 \leq 1$ và $2x_0 + x_1 = 1$ có thể được viết lại dưới dạng ràng buộc tuyến tính chuẩn như sau:

$$\begin{bmatrix} -\infty \\ 1 \end{bmatrix} \leq \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} \leq \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

```
from scipy.optimize import  
    LinearConstraint  
linear_constraint =  
    LinearConstraint([[1, 2], [2,  
        1]], [-np.inf, 1], [1, 1])
```

minimize với ràng buộc

Ràng buộc phi tuyến

$$c(x) = \begin{bmatrix} x_0^2 + x_1 \\ x_0^2 - x_1 \end{bmatrix} \leq \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

với ma trận Jacobian:

$$J(x) = \begin{bmatrix} 2x_0 & 1 \\ 2x_0 & -1 \end{bmatrix},$$

và sự kết hợp tuyến tính của Hessians:

$$H(x, v) = \sum_{i=0}^1 v_i \nabla^2 c_i(x) = v_0 \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix} + v_1 \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix},$$

được sử dụng bởi `NonlinearConstraint` object.

minimize với ràng buộc

```
def cons_f(x):  
    return [x[0]**2 + x[1], x[0]**2 -  
            x[1]]  
def cons_J(x):  
    return [[2*x[0], 1], [2*x[0],  
                            -1]]  
def cons_H(x, v):  
    return v[0]*np.array([[2, 0], [0,  
                                0]]) + v[1]*np.array([[2,  
                                0], [0, 0]])  
from scipy.optimize import  
    NonlinearConstraint  
nonlinear_constraint =  
    NonlinearConstraint(cons_f, -  
                        np.inf, 1, jac=cons_J, hess=  
                        cons_H)
```

minimize với ràng buộc

Giải bài toán

```
x0 = np.array([0.5, 0])
res = minimize(rosen, x0, method=
    'trust-constr', jac=rosen_der
    , hess=rosen_hess,
    constraints=[linear_constraint,
        nonlinear_constraint],
    options={'verbose': 1}, bounds=
        bounds)
print(res.x)
# [0.41494531 0.17010937]
```

Tối ưu hóa hàm một biến với `minimize_scalar`

- Tối ưu hàm $f : \mathbb{R} \rightarrow \mathbb{R}$

Ví dụ: Tìm giá trị nhỏ nhất của hàm $J_1(x)$ gần $x = 5$, giới hạn trong đoạn $[4, 7]$. Ta dùng:

```
from scipy.special import j1
res = minimize_scalar(j1, bounds
                      =(4, 7), method='bounded')
res.x
# Output: 5.33144184241
```

Kết luận: Nghiệm tối ưu là $x_{\min} = 5.3314$.

linprog: Bài toán tối ưu tuyến tính

- Giải bài toán

$$\min c^T x \quad \text{s.t. } Ax \leq b$$

```
from scipy.optimize import  
    linprog  
  
c = [-1, -2]  
A = [[2, 1], [1, 1]]  
b = [20, 16]  
res = linprog(c, A_ub=A, b_ub=b,  
              method='highs')  
print(res.x)
```

least_squares cho fitting dữ liệu

Cho dữ liệu mô phỏng theo công thức:

$$y_i = 3x_i + 2 + \varepsilon$$

với $\varepsilon \sim \mathcal{N}(0, 1)$ Yêu cầu: Tìm các hệ số a và b sao cho đường thẳng:

$$y = ax + b$$

phù hợp nhất với tập dữ liệu (theo tiêu chí bình phương sai số).

least_squares cho fitting dữ liệu

```
import numpy as np
from scipy.optimize import
    least_squares

# Data
x_data = np.linspace(0, 10, 20)
y_data = 3.0 * x_data + 2.0 + np.
    random.normal(size=x_data.
        size)

# Model:  $y = ax + b$ 
def residuals(params, x, y):
    a, b = params
    return a * x + b - y
x0 = [1, 1]
res = least_squares(residuals, x0
    , args=(x_data, y_data))
print(res.x)
```

least_squares cho fitting dữ liệu

Hàm cần tối thiểu hóa là:

$$\min_{a,b} \sum_{i=1}^n (ax_i + b - y_i)^2$$

Hàm least_squares sẽ nội suy để tìm ra cặp (a^*, b^*) sao cho sai số bình phương là nhỏ nhất.

PuLP: Quy hoạch nguyên hỗn hợp

PuLP là một thư viện Python mã nguồn mở dùng để mô hình hóa và giải các bài toán quy hoạch tuyến tính (Linear Programming - LP) và quy hoạch nguyên (Integer Programming - IP).

Ứng dụng:

- Tối ưu hóa chuỗi cung ứng (Supply Chain)
- Lập lịch (Scheduling)
- Lập kế hoạch sản xuất (Production Planning)
- Phân phối tài nguyên (Resource Allocation)
- Và nhiều bài toán liên quan tới tối ưu hóa khác

PuLP: Quy hoạch nguyên hỗn hợp

Tối đa hóa $z = 4x + 3y$ với ràng buộc:

$$\begin{cases} 2x + y \leq 20 \\ 4x - 5y \geq -10 \\ x \geq 0, y \geq 3 \end{cases}$$

PuLP: Quy hoạch nguyên hỗn hợp

```
import pulp
# Initialize the model
model = pulp.LpProblem("
    Maximize_Z", pulp.LpMaximize)
# Variables
x = pulp.LpVariable("x", lowBound
    =0)
y = pulp.LpVariable("y", lowBound
    =3)
# Objective
model += 4 * x + 3 * y
# Constraints
model += 2 * x + y <= 20
model += 4 * x - 5 * y >= -10
# Solve the Problem
model.solve()
# Output
print("x =", x.value())
print("y =", y.value())
print("Value of Z =", pulp.value(
    model.objective))
```

- Bài toán đơn giản:
 - SciPy.optimize cho NLP
 - PuLP cho LP
- Bài toán lồi:
 - CVXPY (cú pháp rõ ràng)
 - CVXOPT cho QP
- Mô hình phức tạp:
 - Pyomo (hỗ trợ nhiều loại bài toán)
 - GEKKO cho hệ thống động

Bài tập

Sử dụng Python để tìm nghiệm tối ưu cho các bài toán sau:

Bài tập 1:

Tìm giá trị nhỏ nhất của hàm mục tiêu:

$$f(x) = 2x_1 + 2x_2 + 3x_3$$

với các ràng buộc:

$$\begin{cases} x_1 + x_2 + x_3 \geq 1 \\ 2x_1 - x_2 + 3x_3 \leq 5 \\ x_1 \geq 0, x_2 \geq 0, x_3 \geq 0 \end{cases}$$

Bài tập

Bài tập 2:

Tìm giá trị nhỏ nhất của hàm mục tiêu:

$$f(x) = x_1 + x_2$$

với các ràng buộc:

$$\begin{cases} x_1^2 + x_2^2 \leq 1 \\ x_1 + x_2 \geq 0 \\ x_1 \geq -1, x_2 \geq -1 \end{cases}$$

Bài tập 3:

Tìm giá trị nhỏ nhất của hàm mục tiêu:

$$f(x) = x_1^2 + 2x_2^2 + 3x_3^2 - 2x_1x_2 - 2x_2x_3$$

- Địa chỉ download phần mềm:
 - CVXPY: <https://www.cvxpy.org/>
 - PuLP: <https://coin-or.github.io/pulp/>
 - Scipy: <https://docs.scipy.org/doc/scipy/tutorial/>
- Tài liệu tham khảo:
 - “*Hands-On Mathematical Optimization with Python*” (2025), K. Postek, A. Zocca, J.A.S. Gromicho, J.C. Kantor, Cambridge University Press.
 - “*Convex Optimization*” (2009), S. Boyd, L. Vandenberghe, Cambridge University Press.

2. Tối ưu hóa với MATLAB

- Là một bộ sưu tập các hàm mở rộng khả năng của MATLAB.
- Bao gồm các hàm cho:
 - Tối ưu hóa không ràng buộc
 - Tối ưu hóa phi tuyến có ràng buộc
 - Lập trình bậc hai và tuyến tính
 - Bình phương tối thiểu phi tuyến và điều chỉnh đường cong
 - Giải hệ phương trình phi tuyến
 - Bình phương tối thiểu tuyến tính có ràng buộc
 - Các thuật toán chuyên biệt cho các bài toán quy mô lớn

Thuật toán tối thiểu hóa

Loại	Ký hiệu	Hàm
Tối thiểu hóa vô hướng	$\min f(a)$ với $a_1 < a < a_2$	finished
Tối thiểu hóa không ràng buộc	$\min f(x)$	fminunc, fminsearch
Lập trình tuyến tính	$\min f^T x$ với $A \cdot x \leq b, Aeq \cdot x = beq, l \leq x \leq u$	linprog
Lập trình bậc hai	$\min \frac{1}{2}x^T Hx + f^T x$ với $A \cdot x \leq b, Aeq \cdot x = beq, l \leq x \leq u$	quadprog

Thuật toán tối thiểu hóa

Loại	Ký hiệu	Hàm
Tối ưu hóa có ràng buộc	$\min f(x)$ với x $c(x) \leq 0, ceq(x) = 0$ $A \cdot x \leq b, Aeq \cdot x = beq, l \leq x \leq u$	fmincon
Đạt mục tiêu	$\min \gamma$ với x, γ $F(x) - w\gamma \leq goal$ $c(x) \leq 0, ceq(x) = 0$ $A \cdot x \leq b, Aeq \cdot x = beq, l \leq x \leq u$	fgoalattain
Minimax	$\min \max\{F_i(x)\}$ với x $c(x) \leq 0, ceq(x) = 0$ $A \cdot x \leq b, Aeq \cdot x = beq, l \leq x \leq u$	fminimax
Tối ưu hóa bán vô hạn	$\min f(x)$ với x $K(x, w) \leq 0$ với mọi w $c(x) \leq 0, ceq(x) = 0$ $A \cdot x \leq b, Aeq \cdot x = beq, l \leq x \leq u$	fseminf

Thuật toán giải phương trình

Loại	Ký hiệu	Hàm
Phương trình tuyến tính	$C \cdot x = d$	<code>\ (slash)</code>
Phương trình phi tuyến một biến	$f(a) = 0$	<code>fzero</code>
Phương trình phi tuyến	$F(x) = 0$	<code>fsolve</code>

Thuật toán bình phương tối thiểu

Loại	Ký hiệu	Hàm
Bình phương tối thiểu tuyến tính	$\min_x \ C \cdot x - d\ _2^2$, m phương trình, n biến	\ (slash)
Không âm	$\min_x \ C \cdot x - d\ _2^2$ với $x \geq 0$	lsqnonneg
Bình phương tối thiểu tuyến tính có ràng buộc	$\min_x \ C \cdot x - d\ _2^2$ với $A \cdot x \leq b$, $Aeq \cdot x = beq$, $l \leq x \leq u$	lsqlin
Bình phương tối thiểu phi tuyến	$\min_x \frac{1}{2} \ F(x)\ _2^2 = \frac{1}{2} \sum_i F_i(x)^2$ với $l \leq x \leq u$	lsqnonlin
Điều chỉnh đường cong phi tuyến	$\min_x \frac{1}{2} \ F(x, xdata) - ydata\ _2^2$ với $l \leq x \leq u$	lsqcurvefit

Triển khai Optimization Toolbox

- Hầu hết các hàm tối ưu hóa yêu cầu định nghĩa một file M-file chứa hàm số f cần được tối thiểu hóa.
- Tối đa hóa được thực hiện bằng cách cung cấp các hàm với $-f$.
- Các tùy chọn tối ưu hóa được truyền vào các hàm thay đổi các tham số tối ưu hóa.
- Các tham số tối ưu hóa mặc định có thể được thay đổi thông qua một cấu trúc tùy chọn.

Tối ưu hóa không ràng buộc

- Xét bài toán tìm một tập giá trị $[x_1, x_2]^T$ sao cho:

$$\min_x f(x) = e^{x_1} (4x_1^2 + 2x_2^2 + 4x_1x_2 + 2x_2 + 1)$$

$$x = [x_1 \quad x_2]^T$$

Các bước thực hiện

- Tạo một file M-file trả về giá trị hàm số (Hàm mục tiêu).
- Gọi nó là `objfun.m`.
- Sau đó, gọi hàm tối ưu hóa không ràng buộc.
- Sử dụng `fminunc`.

Bước 1 - Hàm mục tiêu

```
function f = objfun(x)
% x = [x_1 x_2]^T
f = exp(x(1)) * (4*x(1)^2 + 2*x(2)^2 + 4*
    x(1)*x(2) + 2*x(2) + 1);
```

Bước 2 - Gọi hàm tối ưu hóa

```
x0 = [-1, 1];  
options = optimset('LargeScale', 'off');  
[xmin, feval, exitflag, output] = fminunc  
    (@objfun, x0, options);
```

Kết quả

- `xmin = [0.5000, -1.0000]`
- `feval = 1.3028e-010`
- `exitflag = 1`
- `output:`
 - iterations: 7
 - funcCount: 40
 - stepsize: 1

Thông tin thêm về `fminunc` - Input

- **fun**: Trả về một hàm của hàm mục tiêu.
- **x0**: Bắt đầu với một giá trị khởi tạo. Giá trị khởi tạo phải là một vector có kích thước bằng số biến thiết kế.
- **option**: Để thiết lập một số tham số tối ưu hóa.
- **P1, P2, ...**: Để truyền các tham số bổ sung.

Thông tin thêm về fminunc - Output

- **xmin**: Vector của điểm tối thiểu (điểm tối ưu). Kích thước của vector bằng số biến thiết kế.
- **feval**: Giá trị của hàm mục tiêu tại điểm tối ưu.
- **exitflag**: Một giá trị cho biết liệu quá trình tối ưu hóa có kết thúc thành công hay không. (hội tụ nếu > 0)
- **output**: Cấu trúc này cung cấp thêm chi tiết về quá trình tối ưu hóa.
- **grad**: Giá trị gradient tại điểm tối ưu.
- **hessian**: Giá trị Hessian tại điểm tối ưu.

Thiết lập tùy chọn - optimset

- Các hàm trong Optimization Toolbox có một bộ tham số tối ưu hóa mặc định.
- Tuy nhiên, toolbox cho phép bạn thay đổi một số tham số này, ví dụ:
 - Độ chính xác (tolerance)
 - Kích thước bước (step size)
 - Giá trị gradient hoặc Hessian
 - Số lần lặp tối đa (max number of iterations)
- Có một danh sách các tính năng có sẵn, ví dụ:
 - Hiển thị giá trị tại mỗi lần lặp
 - So sánh gradient hoặc Hessian do người dùng cung cấp
- Bạn cũng có thể chọn thuật toán mà bạn muốn sử dụng.

Thiết lập tùy chọn - optimset

- Gõ **help optimset** trong cửa sổ lệnh, một danh sách các tùy chọn thiết lập có sẵn sẽ được hiển thị.
- Cách đọc? Ví dụ:

Thiết lập tùy chọn hữu ích

- **Display:** Mức độ hiển thị [off | iter | notify | final]
- **MaxIter:** Số lần lặp tối đa cho phép [số nguyên dương]
- **TolCon:** Độ chính xác kết thúc đối với vi phạm ràng buộc [số thực dương]
- **TolFun:** Độ chính xác kết thúc đối với giá trị hàm [số thực dương]
- **TolX:** Độ chính xác kết thúc đối với X [số thực dương]

So sánh `fminunc` và `fminsearch`

- `fminunc` sử dụng thông tin gradient và Hessian.
- Hai chế độ:
 - Large-Scale: Newton phản xạ nội bộ
 - Medium-Scale: BFGS (quasi-Newton)
- Không phù hợp với các hàm có độ gián đoạn cao.
- Có thể chỉ cho ra nghiệm cục bộ.

So sánh `fminunc` và `fminsearch`

- `fminsearch` thường kém hiệu quả hơn `fminunc` đối với các bài toán có bậc lớn hơn hai. Tuy nhiên, khi bài toán có tính chất *gián đoạn* cao, `fminsearch` có thể mạnh mẽ hơn.
- Đây là một phương pháp tìm kiếm trực tiếp không sử dụng gradient số hoặc giải tích như trong `fminunc`.
- Hàm này có thể chỉ cho ra các nghiệm cục bộ.

Tối ưu hóa có ràng buộc

$$\min f(x) \quad \text{với} \quad c(x) \leq 0, \text{ceq}(x) = 0$$

$$A \cdot x \leq b, Aeq \cdot x = beq, lb \leq x \leq ub$$

Trong đó x , b , beq , lb , và ub là các vector, A và Aeq là các ma trận, $c(x)$ và $ceq(x)$ là các hàm trả về vector, và $f(x)$ là một hàm trả về một giá trị vô hướng. $f(x)$, $c(x)$, và $ceq(x)$ có thể là các hàm phi tuyến.

- Bài toán:

$$\min_x f(x) = -x_1 x_2 x_3$$

- Với các ràng buộc:

$$2x_1^2 + x_2 \leq 0$$

$$-x_1 - 2x_2 - 2x_3 \leq 0$$

$$x_1 + 2x_2 + 2x_3 \leq 72$$

$$0 \leq x_1, x_2, x_3 \leq 30$$

Ma trận và vector

$$A = \begin{bmatrix} -1 & -2 & -2 \\ 1 & 2 & 2 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 72 \end{bmatrix}$$

$$LB = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad UB = \begin{bmatrix} 30 \\ 30 \\ 30 \end{bmatrix}$$

Hàm ràng buộc phi tuyến

Đối với

$$2x_1^2 + x_2 \leq 0$$

Tạo một hàm có tên `nonlcon` trả về 2 vector ràng buộc `[C, Ceq]`

```
function [C, Ceq] = nonlcon(x)
C = 2*x(1)^2 + x(2);
Ceq = [];
```

Ví dụ (Tiếp theo)

Initial guess (3 design variables)

`x0=[10;10;10];`

`A=[-1 -2 -2;1 2 2];`

`B=[0 72]';`

`LB = [0 0 0]';`

`UB = [30 30 30]';`

`[x,feval]=fmincon(@myfun,x0,A,B,[ ],[ ],LB,UB,@nonlcon)`

$$A = \begin{bmatrix} -1 & -2 & -2 \\ 1 & 2 & 2 \end{bmatrix}, B = \begin{bmatrix} 0 \\ 72 \end{bmatrix}$$

$$LB = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, UB = \begin{bmatrix} 30 \\ 30 \\ 30 \end{bmatrix}$$

CAREFUL!!!

`fmincon(fun,x0,A,B,Aeq,Beq,LB,UB,NONLCON,options,P1,P2,...)`

Ví dụ (Tiếp theo)

Optimization terminated successfully:

Magnitude of directional derivative in search direction less than 2*options.TolFun and maximum constraint violation is less than options.TolCon

Active Constraints:

2
9

x =

0.00050378663220

0.000000000000000

30.000000000000000

feval =

-4.657237250542452e-035

$$-x_1 - 2x_2 - 2x_3 \leq 0 \quad \text{Const. 1}$$

$$x_1 + 2x_2 + 2x_3 \leq 72 \quad \text{Const. 2}$$

$$\text{Const. 3} \quad 0 \leq x_1 \leq 30 \quad \text{Const. 5}$$

$$\text{Const. 4} \quad 0 \leq x_2 \leq 30 \quad \text{Const. 6}$$

$$\text{Const. 7} \quad 0 \leq x_3 \leq 30 \quad \text{Const. 8}$$

$$2x_1^2 + x_2 \leq 0 \quad \text{Const. 9}$$

Sequence: A, B, Aeq, Beq, LB, UB, C, Ceq

Kết quả

- $x = [0.0005; 0.0000; 30.0000]$
- $\text{feval} = -4.6572\text{e-}035$

Tối ưu tuyến tính

- MATLAB sử dụng định dạng sau cho bài toán tối ưu tuyến tính:

$$\min f^T x \quad \text{với} \quad A \cdot x \leq b, Aeq \cdot x = beq, l \leq x \leq u$$

- Giải bằng lệnh:

$$x = \text{linprog}(f, A, b, Aeq, beq, l, u)$$

- Bài toán:

$$\max 4x_1 + 2x_2 + x_3$$

- Với các ràng buộc:

$$2x_1 + x_2 \leq 1$$

$$x_1 + 2x_3 \leq 2$$

$$x_1 + x_2 + x_3 = 1$$

$$0 \leq x_1 \leq 1, 0 \leq x_2 \leq 1, 0 \leq x_3 \leq 2$$

Ví dụ

- Chuyển đổi sang định dạng MATLAB:

$$f = - \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix}$$

$$A_{eq} = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}, \quad b_{eq} = \begin{bmatrix} 1 \end{bmatrix}$$

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad l = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad u = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$$

- Giải bài toán:

```
x = linprog(f, A, b, Aeq, beq, l, u)
```

- $x = [0.5000; 0.0000; 0.5000]$

Bài tập

Sử dụng MATLAB để tìm nghiệm tối ưu cho các bài toán sau:

Bài tập 1.

Tìm giá trị nhỏ nhất của hàm mục tiêu:

$$f(x) = 2x_1 + 2x_2 + 3x_3$$

với các ràng buộc:

$$\begin{cases} x_1 + x_2 + x_3 \geq 1 \\ 2x_1 - x_2 + 3x_3 \leq 5 \\ x_1 \geq 0, x_2 \geq 0, x_3 \geq 0 \end{cases}$$

Bài tập

Bài tập 2.

Tìm giá trị nhỏ nhất của hàm mục tiêu:

$$f(x) = x_1 + x_2$$

với các ràng buộc:

$$\begin{cases} x_1^2 + x_2^2 \leq 1 \\ x_1 + x_2 \geq 0 \\ x_1 \geq -1, x_2 \geq -1 \end{cases}$$

Bài tập 3.

Tìm giá trị nhỏ nhất của hàm mục tiêu:

$$f(x) = x_1^2 + 2x_2^2 + 3x_3^2 - 2x_1x_2 - 2x_2x_3$$

3. Một số phần mềm khác: CPLEX, Gurobi, Excel Solver

Bài tập: Tìm hiểu và trình bày ngắn gọn về các phần mềm trên.